Software Deployment

What is Deployment?

Deployment is the process of **pushing changes or updates** from one environment to another. It ensures that software moves through different stages before reaching the end user.

Environments

An **environment** refers to a setup where software is developed, tested, or used.

- **Development Environment:** This is where developers write and test their code.
- **Production (Live) Environment:** This is where the final version of the software is accessible to end-users or clients.

For example, when you code on your local computer, that's the **Development Environment**. But when a client accesses your website or software online, they are using the **Production Environment**.

Deployment Model

Deployment follows a **left-to-right model**, meaning software moves from **coding** to the **client's hands** through several stages.

1. Local Environment

This is where the initial development happens. Developers test features on their own systems.

👉 Example: If you set up a messaging service on your PC and send a message that appears on your phone, that entire setup is your **local environment**.

2. Development Environment

Once a feature works properly in the **local environment**, it is moved to a **development server** for further testing.

🔹 **Note:** In many cases, the **local and development environments are the same** for individual developers.

🔹 **This environment may not include all features or microservices** of the full application.

3. Staging Environment

The **staging environment** is a replica of the **production environment** but is only used for testing.

➡️ It contains all microservices, plus any newly added features.
➡️ **Quality Assurance (QA) teams** perform tests here before release.

Example: Before launching an app update, the complete system is tested in staging to ensure all features work as expected.

4. Production (Live) Environment

If everything functions properly in **staging**, the application is deployed to the **production environment**. This is where real users interact with the software.

Deployment means moving updates from one environment to another until the final product is live.

Deployment Process Flow

- Create a Software Deployment Plan
- Develop the software
- Test changes in staging
- Deploy updates to the live environment
- Monitor for any issues

Types of Deployment

1. Deployment of Meta-Data

Includes code, CSS, configuration files
Controlled by developers

2. Deployment of Content

Includes images, videos, text
Can be uploaded by content creators

Best Practices for Deployment

- **Use Git for version control.** Distributed version control (no central dependency). Enables **collaboration, branching, and rollback features**. **Git** is the most widely used version control system, but there are alternatives as well:
    - Mercurial (Hg): Similar to Git but simpler and more user-friendly.
    - Apache Subversion (SVN): Centralized version control, often used in legacy systems.
    - Perforce: Used in large enterprises with complex codebases (e.g., game development).
- **Work in branches** to manage code efficiently: Using branches in Git (or any version control system) allows developers to work on new features without affecting the main codebase.
    - Feature branching, Name base branching, Hotfix branches, release branches.
- **Review changes before going live.** Code reviews help maintain quality, catch errors early, and ensure best practices are followed.

    ### Industry-Standard Code Review Process:

    Use Pull Requests (PRs) / Merge Requests (MRs):

    - Before merging changes, create a PR in GitHub/GitLab/Bitbucket.
    - Assign reviewers who provide feedback before approving.

    Follow the 4-Eyes Principle: At least **one other developer** must review the changes before merging.

    Use Code Review Tools:

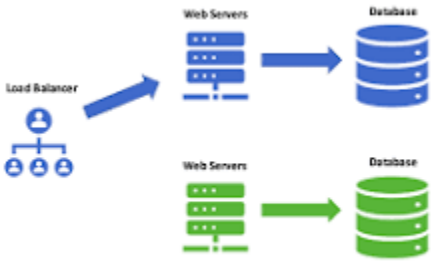
    - **GitHub/GitLab/Bitbucket Review System**
    - **SonarQube** – for static code analysis
    - **Crucible** – enterprise-level code review tool
- **Follow a deployment schedule** based on user activity & developer availability. Deploying at the right time reduces risks and minimizes impact.

    Best Practices for Scheduling:

    - Deploy during **off-peak hours** to minimize user disruption.
    - Ensure **developers and support teams** are available post-deployment.
    - Use **progressive rollouts** (e.g., deploy to a small percentage of users first).

    Example Deployment Strategies:

- **Blue-Green Deployment:** Two **identical** environments. One environment (blue) is running the **current** application version and one environment (green) is running the **new** application version. It simplifies the rollback process if a deployment fails.



  Once testing has been completed on the green environment, live application traffic is directed to the green environment and the blue environment is deprecated.

- **Canary Deployment:** Release updates to a small group of users before a full rollout.
- **Rolling Deployment:** Update servers gradually instead of all at once.

- **Set different permission levels** – Not all developers need deployment access. Limiting access ensures **security and accountability**.

  Use Role-Based Access Control (RBAC):

  - Developers: Can push code but **cannot deploy**.
  - Release Managers: Can **approve and trigger** deployments.
  - Admins: Have full control but **use access sparingly**.
- **Stay calm, even if something goes wrong!**
  - **Use Monitoring Tools**: Tools like Datadog, Prometheus, New Relic help track errors in real time.
  - **Set Up Automated Rollbacks**: If a deployment fails, automatically revert to the last working version.
  - **Have an Incident Response Plan**: Know who to contact and what steps to take in case of failure.

Software Deployment Methodology: DevOps

DevOps is a **methodology and set of best practices** aimed at making deployment faster and more efficient. It was introduced to address the disconnect primarily between the **development**, **operations**, and **quality assurance** teams.

DevOps Lifecycle: 7 Phases Explained in Detail with Examples

Release vs. Deployment

A **release** refers to a **new version of the software** that includes **new features, bug fixes, or improvements**. It is typically assigned a **version number** (e.g., **4.0.1** → **4.0.2**).

✅ **Example:**

- Suppose your current application version is **4.0.1**.
- You develop a **new feature** or fix a **bug** and update the version to **4.0.2**.
- This **4.0.2 version is now ready** for use but **has not yet been deployed**.
- It remains in a **staging or testing environment** until it is approved for deployment.

Think of **release** as **preparing a new version of an app**—but it's not yet available for the users.

A **deployment** is the process of **moving the released software into a live environment (production) so that users can access it**.

✅ **Example:**

- The **4.0.2 version** you prepared is now stable.
- You decide to **deploy** it to the production server.
- Now, users can **see and use the new version** of the software.